

Введение в React

Начало пути к современным
веб-приложениям



Компоненты



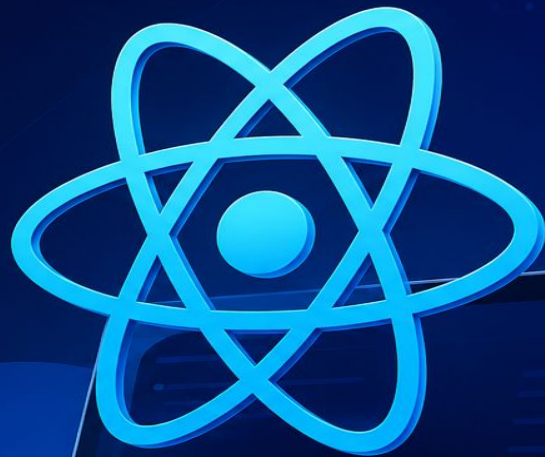
Быстродействие



Реактивность



Переиспользование



JS

{ }

```
function Hello() {  
  return (  
    <div className="title">  
      <h1>Привет, React!</h1>  
    </div>  
  );  
}
```

Знакомство

Меня зовут **Lad**, я ваш преподаватель и ментор на этом курсе.



Преподаватель



Ведущий Frontend разработчик



Моя специализация:
React, TS, Next.js



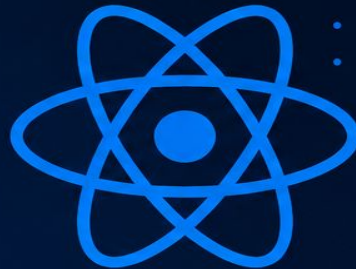
Пройден полный цикл обучения в академии,
совмещая работу, учебу и стажировку



За спиной более **8 проектов.**



Опыт в разработке более **3,5 лет**



Работал над **проектами**
разной сложности



Постоянно развиваюсь
и изучаю **новые технологии**

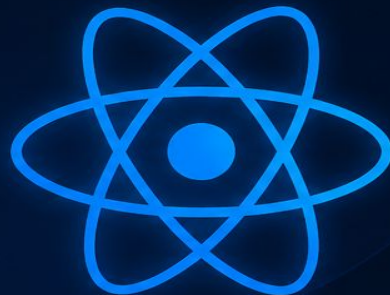


Помогаю разработчикам расти
и **достигать своих целей**



Как будет проходить наш курс

Практико-ориентированный курс с акцентом на реальные задачи и современные инструменты.
Теория → Практика → Проекты



01 Теория

Изучаем ключевые концепции и инструменты для разработки на React и TypeScript.



02 Практика

Закрепляем знания на практических заданиях и небольших проектах.



03 Проекты

Работаем над реальными проектами, применяя полученные знания на практике.



04 Рост и развитие

Улучшаем навыки, осваиваем новые технологии и становимся уверенными разработчиками.

React — что это?



React — это библиотека для создания пользовательских интерфейсов.



Зачем нужен:

React помогает строить динамичные и интерактивные веб-приложения, обновляя только те части интерфейса, которые изменяются, что делает приложения быстрее и более отзывчивыми.



Проблемы, которые решает:



• UI обновления

React эффективно обновляет только изменённые части страницы, минимизируя перезагрузку.



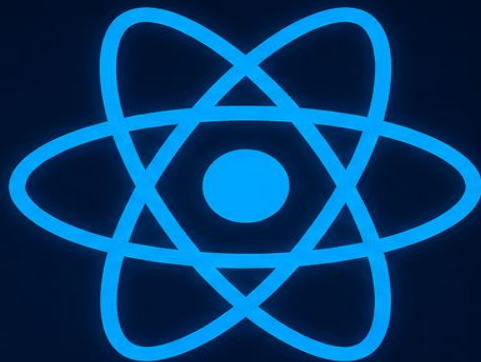
• Компонентный подход

Позволяет строить интерфейсы из переиспользуемых компонентов, что улучшает структуру и поддержку кода.



• Управление состоянием

Предоставляет механизмы для управления состоянием компонентов, что упрощает работу с динамичными данными.



React делает разработку современных веб-приложений **проще, быстрее и масштабируемее**.



Быстро



Удобно



Масштабируемо



Надёжно

npm — что это?



npm (Node Package Manager) — это менеджер пакетов для JavaScript, который используется для управления зависимостями в проектах, работающих с Node.js.



Основные моменты:



• Управление зависимостями:

Пакеты добавляются через **npm install**, а список зависимостей хранится в файле **package.json**.



• Глобальная и локальная установка:

Можно установить пакеты локально (для проекта) или глобально (для использования по всей системе).



• Скрипты:

В **package.json** можно прописывать команды для автоматизации задач, которые выполняются через **npm run**.



• npm registry:

Пакеты хранятся в онлайн-репозитории **npmjs.com**.



Пример команд:

npm init

— создаёт package.json.

npm install

— устанавливает зависимости.

npm install <пакет>

— устанавливает конкретный пакет.

npm uninstall <пакет>

— удаляет конкретный пакет.

npm run <скрипт>

— выполняет скрипт из package.json.

npm install -g <пакет>

— устанавливает пакет глобально.



npm — важный инструмент для работы с зависимостями в JavaScript-проектах.



Где используется:



Node.js



React



TypeScript



и другие

npm упрощает установку, обновление и управление библиотеками, инструментами и зависимостями в проектах.

Версионирование в npm

Версионирование в npm использует **семантическое версионирование** (SemVer).



С помощью версионирования можно **контролировать совместимость** пакетов в проекте.



Обновление зависимостей



```
npm install <пакет>@<версия>
```

Устанавливает конкретную версию.



```
npm outdated
```

Показывает устаревшие пакеты.



```
npm update
```

Обновляет зависимости до разрешённых версий.



Всегда проверяйте **документацию** пакета и совместимость версий, чтобы избежать проблем в проекте.

SemVer: **MAJOR**.**MINOR**.**PATCH**



Основные изменения, несовместимые с предыдущими версиями.

Пример:

2.0.0



Новые функции, совместимые с предыдущими версиями.

Пример:

1.2.0



Исправления ошибок, не влияющие на функциональность.

Пример:

1.2.3



Указания версий:

^1.2.3

Обновление до минорных и патч-версий (1.x.x).

~1.2.3

Обновление только до патч-версий (1.2.x).

1.2.3

Точная версия.

Любая версия.



Модули (import / export)

import и export — это синтаксис для работы с модулями в JavaScript, который позволяет разделять код на отдельные файлы и легко обмениваться функциональностью между ними.



Варианты использования

01



Named Export (именованный экспорт)

Экспортируются конкретные элементы (функции, переменные, классы).

02



Default Export (экспорт по умолчанию)

Экспортируется один элемент как "по умолчанию".

03



Экспорт с переименованием

Можно переименовывать элементы при импорте.

Примеры

В файле: `utils.js`

```
export const sum = (a, b) => a + b;
export const PI = 3.14;
export function greet(name) {
  return `Привет, ${name}!`;
}
```

В файле: `Button.js`

```
export default function Button(text) {
  return <button>{text}</button>;
}
```

В файле: `math.js`

```
export const multiply = (a, b) => a * b;
export const divide = (a, b) => a / b;
```

В другом файле: `app.js`

```
import { sum, PI, greet } from './utils.js';
console.log(sum(2, 3));
console.log(PI);
console.log(greet('Lad'));
```

В другом файле: `app.js`

```
import Button from './Button.js';
export default function App() {
  return <Button text="Нажми меня" />;
}
```

В другом файле: `calc.js`

```
import { multiply as mul, divide as div }
  from './math.js';
console.log(mul(3, 4));
console.log(div(10, 2));
```



Важные моменты



Named Export:

можно экспортировать несколько элементов.



Default export:

только один элемент на файл.



Модули import и export помогают структурировать код, делают его удобным для масштабирования и повторного использования.



TypeScript — знакомство с типами

TypeScript — это надстройка над JavaScript, добавляющая статическую типизацию и другие улучшения, которые делают код более предсказуемым и безопасным.



Зачем нужен?

- ✓ **Типизация:** обнаружение ошибок на этапе разработки, до выполнения кода.
- ✓ **Современные возможности JS:** поддержка ES6+ и транпиляция в совместимый код.
- ✓ **Лучший опыт разработки:** автодополнение, подсказки и рефакторинг в IDE.



Ключевые возможности TypeScript



Обнаружение ошибок
на этапе разработки

Меньше багов
в продакшене.



Интерфейсы
и типы

Структурирование
данных и контрактов.



Дженерики

Создание гибких
и переиспользуемых
компонентов.



Безопасность
и масштабируемость

Более стабильный
и поддерживаемый код.

TS

Типы данных в TypeScript

T

string

строка

```
let name: string = "React";
```

123

number

число

```
let age: number = 25;
```

✓

boolean

булево значение

```
let isActive: boolean = true;
```

*

any

любой тип

```
let value: any = 42;
```

⊘

null

пустое значение

```
let empty: null = null;
```

⊖

undefined

неопределенное значение

```
let u: undefined = undefined;
```

○

void

нет возвращаемого значения

```
function log(): void {}
```

{ }

object

объект

```
let obj: object = {};
```

</>

Пример

```
1 interface User {
2   id: number;
3   name: string;
4   isActive: boolean;
5 }
6
7 const user: User = {
8   id: 1,
9   name: "React Developer",
9   isActive: true
10 };
```



TypeScript помогает
писать более надёжный
и понятный код,
снижает количество
ошибок и ускоряет
разработку больших
приложений.